UNIVERSIDAD AUTONÓMA DEL ESTADO DE MÉXICO

Centro Universitario UAEM Nezahualcóyotl

Carrera:

Ingeniería en Sistemas Inteligentes

Título del proyecto: SignAI

Profesor: Víctor Antonio Flores Lezama

EQUIPO:
Integrante 1. Arias Pérez Casandra Guadalupe
Integrante 2. Delis Monterrubio Esli Michelle

ISI TURNO: MATUTINO

SEMESTRE: noveno

GRUPO “S9”

Índice

Introducción 3
Contenido 4
UI (interfaz de usuario) 5
Herramientas y materiales 8
Repositorio (GitHub) 9
Estado del arte 13
Desarrollo del software 16
Big data 24
Capturas beta 25
UX (Experiencia del usuario) 28
Referencias bibliográficas 33
Resumen del proyecto 37

Introducción

La lengua de señas es la forma de comunicación que usan principalmente las personas sordas o con dificultades auditivas. El problema aparece cuando quieren conversar con alguien que no la conoce: sin un intérprete a la mano, entenderse puede ser lento y complicado. Según la Federación Mundial de Sordos, citada por Rastgoo et al. (2021), existen más de 300 lenguas de señas y alrededor de 70 millones de personas sordas que las utilizan.

Para dar un primer paso en esa dirección desarrollamos SignAI, una aplicación de escritorio que reconoce letras de la lengua de señas a partir de la posición de la mano frente a la cámara de la computadora. La persona forma una letra con la mano y al momento, la pantalla le muestra cuál reconoció el sistema. No hacen falta guantes ni sensores especiales: basta con una cámara web.

Detrás de esa respuesta hay dos herramientas principales. OpenCV toma las imágenes de la cámara y MediaPipe ubica en ellas 21 puntos clave de la mano, como las puntas de los dedos, los nudillos y la muñeca. Con esos puntos, un modelo de aprendizaje automático decide qué letra se está formando. Ese modelo aprende con ejemplos, de forma parecida a como una persona aprende a reconocer letras después de verlas muchas veces: el administrador graba muestras de cada seña, indica a qué letra corresponde cada una y el sistema se entrena con ellas. El sistema aprende de los ejemplos, lo que permite mejorar y agregar letras nuevas con solo grabar más muestras. El resultado se muestra en una ventana sencilla donde la persona puede activar la cámara, hacer una seña y ver la letra que el sistema reconoce.

El alcance de esta etapa es reconocer letras individuales del abecedario de la Lengua de Señas Mexicana (LSM) que se forman con la mano quieta. El reconocimiento de palabras, de frases completas y de letras que requieren movimiento queda fuera por ahora y se plantea como trabajo futuro.

Esta documentación está escrita pensando en quien va a recibir y utilizar el sistema, no en quien lo programó. Por eso evitamos el lenguaje técnico siempre que fue posible y cuando un término es necesario, lo explicamos con palabras sencillas.

Contenido

Este apartado reúne lo que se necesita saber para entender cómo se ve, con qué está hecho y dónde vive SignAI. Se divide en tres partes y cada una responde a una pregunta sencilla:

- **Interfaz de usuario (UI):** ¿cómo se verá el programa y cómo se recorrerá? Incluye un diagrama con las pantallas y los pasos que sigue cada persona, ya sea el administrador (quien prepara el sistema) o el usuario libre (quien solo lo utiliza).
- **Herramientas y materiales:** ¿con qué está hecho? Presenta los programas y componentes que hacen funcionar el sistema, explicados con palabras sencillas.
- **Repositorio:** ¿dónde se guarda el código y cómo está ordenado? El repositorio es una carpeta en línea (en GitHub) donde se conservan el programa y el historial de todos sus cambios.

Después de este apartado se presentan el estado del arte (qué se ha hecho antes en este tema), el desarrollo del software (cómo funciona por dentro), una nota sobre big data, las capturas de la versión beta, la experiencia de uso paso a paso, las referencias consultadas y un resumen final del proyecto.

UI (interfaz de usuario)

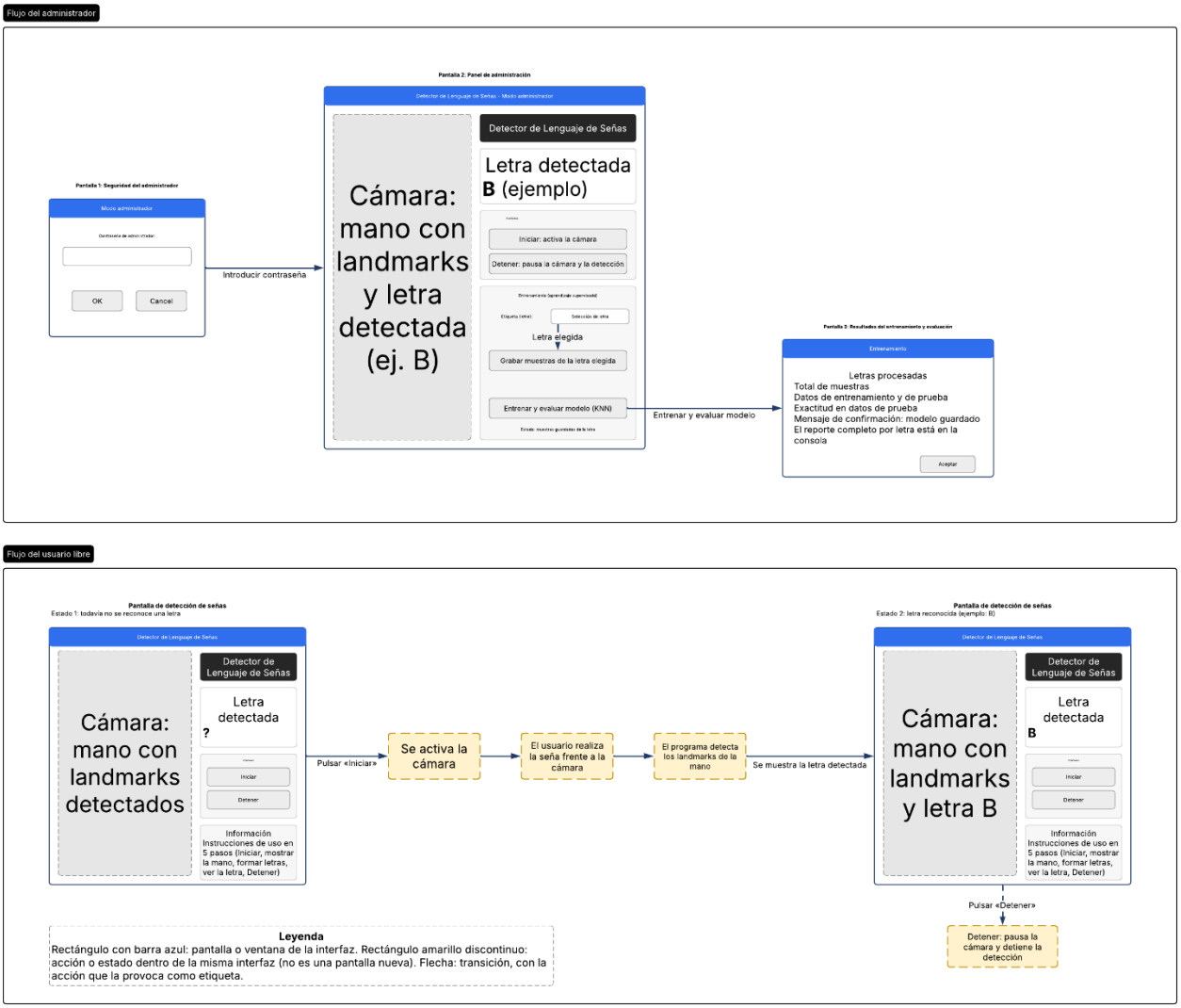


Figura 1. "Diagrama de experiencia de usuario y navegación del sistema", elaboración propia en Lucidchart (2026)

Cómo se usa el sistema: diagrama de experiencia de usuario

Este diagrama permite entender qué ve y qué hace cada persona al usar el sistema de reconocimiento de lengua de señas. Cada recuadro es una pantalla o una ventana del programa y cada flecha muestra a dónde se pasa después de hacer algo. Lo que está escrito sobre la flecha es justo eso: la acción que lleva de una pantalla a otra.

El sistema se usa de 2 maneras distintas así que el diagrama las divide. Arriba está el recorrido del administrador quien prepara el sistema y abajo el del usuario quien simplemente lo usa.

Administrador

Empieza con una contraseña, antes de entrar a sus herramientas el administrador escribe su contraseña y pulsa "OK". Si es la correcta, el sistema lo deja pasar al panel de administración y si se equivoca aparece una ventanita con el mensaje "Contraseña incorrecta" y al pulsar "Aceptar" se cierra sin mostrar otra pantalla. Si en lugar de escribirla pulsa "Cancel" pasa lo mismo: la ventana simplemente se cierra.

Una vez dentro, el administrador encuentra todo en una sola pantalla: el panel de administración. Ahí puede encender la cámara con "Iniciar" y pausarla con "Detener", además hace que el sistema deje de buscar letras. También ahí es donde se le enseña al sistema a reconocer las señas: primero se elige en una lista la letra que se quiere practicar (por ahora, solo las que se hacen con la mano quieta), luego se pulsa "Grabar muestras" y se hace esa seña frente a la cámara para que el sistema guarde ejemplos de cómo se ve. Cuando ya hay suficientes ejemplos, el botón "Entrenar y evaluar modelo" hace que el sistema aprenda de ellos y al mismo tiempo compruebe qué tan bien está reconociendo las letras.

Al terminar, se abre una ventana con el resumen el cual dice cuántas letras y cuántas muestras se usaron, cuántas sirvieron para que el sistema aprendiera y cuántas para ponerlo a prueba y qué porcentaje de aciertos tuvo en esa prueba. También confirma que el modelo quedó guardado en el data set para poder usarlo después sin volver a entrenar.

Usuario

Aquí todo es más sencillo. Quien quiera usar el sistema no necesita contraseña: entra directo a la pantalla de detección, que solo tiene dos botones: "Iniciar" y "Detener", y que funcionan igual que los del administrador.

Al abrir la pantalla, en el lugar donde debe aparecer la letra se ve un signo de interrogación, porque el sistema todavía no ha reconocido nada. La persona pulsa "Iniciar" con lo que se enciende la cámara, y hace una seña frente a ella. El programa localiza los puntos principales de la mano, como

los dedos y la palma, y con eso muestra en el panel de la derecha la letra que identificó. Ojo, en el diagrama se usa la B como ejemplo, pero eso no quiere decir que sea la única que reconoce.

Un detalle útil: el paso del signo de interrogación a la letra no cambia de pantalla. Es la misma pantalla que se va actualizando, por eso en el diagrama aparece como dos momentos de una misma interfaz. Cuando la persona termina, pulsa “Detener“, la cámara se pausa y el sistema deja de detectar.

Herramientas y materiales

SignAI está hecho principalmente con Python, un lenguaje de programación muy usado en visión por computadora y aprendizaje automático. Para no empezar desde cero, el proyecto se apoya en librerías: paquetes de código ya hechos y probados que resuelven tareas concretas algo así como herramientas que se reutilizan en lugar de fabricarlas de nuevo.

Estas son las principales, explicadas lo más sencillo posible:

- **Python:** el lenguaje en el que está escrito el programa.
- **OpenCV:** se encarga de la cámara: toma las imágenes de la cámara web y las prepara para analizarlas.
- **MediaPipe:** ubica la mano en la imagen y marca sus 21 puntos de referencia (puntas de los dedos, nudillos y muñeca). Es lo que se ve como un "esqueleto" dibujado sobre la mano.
- **Scikit-learn:** contiene el modelo de aprendizaje automático (Random Forest) que decide qué letra se está formando.
- **NumPy:** hace con rapidez los cálculos con números.
- **Joblib:** guarda el modelo ya entrenado en un archivo para poder usarlo después sin volver a entrenarlo.
- **Cryptography:** genera las claves y la firma digital que protegen al modelo de alteraciones (se explica en el apartado de seguridad).
- **Tkinter:** dibuja la ventana, los botones y los paneles de la aplicación.
- **Pillow (PIL):** ayuda a mostrar dentro de la ventana las imágenes que llegan de la cámara.
- **Threading, Queue y Time** (vienen incluidos en Python): hacen que el video se procese "en segundo plano" para que la ventana no se trabe y llevan el control de los tiempos.

Además de las herramientas, el sistema necesita un conjunto de datos propio: ejemplos de señas grabados por el administrador desde la propia aplicación, cada uno marcado con la letra que representa. Con ellos se entrena el modelo y después se comprueba qué tan bien aprendió.

Repositorio (GitHub)

El código de SignAI está dividido en piezas pequeñas, cada una con una tarea concreta. Así si algún día hay que corregir o mejorar algo se sabe dónde buscar sin tocar lo demás.

Enlace del repositorio: https://github.com/cas-a14/Senas_traductor

A continuación se explica cada grupo de archivos:

1. Qué es y dónde está

Un repositorio es una carpeta en línea donde se guarda el código de un proyecto junto con el historial de todos sus cambios. SignAI se guarda en GitHub en el repositorio **Senas_traductor**.

En la captura se ve la carpeta principal del proyecto: el nombre del repositorio, la lista de archivos explicados en este apartado y el último cambio guardado.

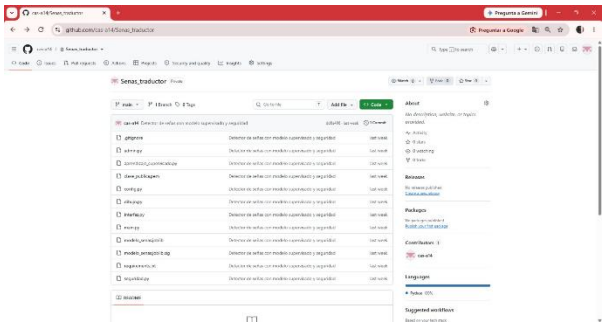


Figura 2. "Página principal del repositorio en GitHub", elaboración propia (2026)

2. Archivos para abrir y usar el programa

main.py abre el programa para el usuario (solo detección de letras). **admin.py** abre el programa para el administrador y pide contraseña.

interfaz.py dibuja la ventana, los botones y el video. **aprendizaje_supervisado.py** se encarga de aprender y reconocer letras.

dibujo.py dibuja el recuadro y los puntos sobre la mano.

config.py reúne los ajustes generales del sistema.

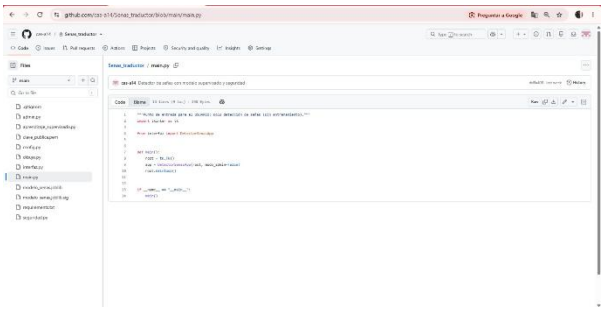


Figura 3. "Código del programa del administrador (admin.py) en GitHub", elaboración propia (2026)

3. Modelo y seguridad

modelo_senas.joblib es el modelo ya entrenado como tal, se guarda en el repositorio para que no haga falta entrenar nada para empezar a usarlo.

modelo_senas.joblib.sig es la firma que garantiza que el modelo no fue alterado y **clave_publica.pem** permite comprobar esa firma.

seguridad.py maneja la contraseña, las claves y la firma.

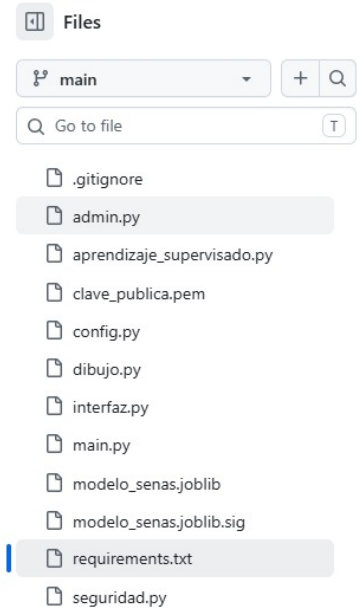


Figura 4. "Archivos del modelo y de seguridad en el repositorio", elaboración propia (2026)

4. Dependencias

requirements.txt es la lista de librerías que hay que instalar para que el programa funcione. Con un solo comando (pip install -r requirements.txt) se instalan todas.

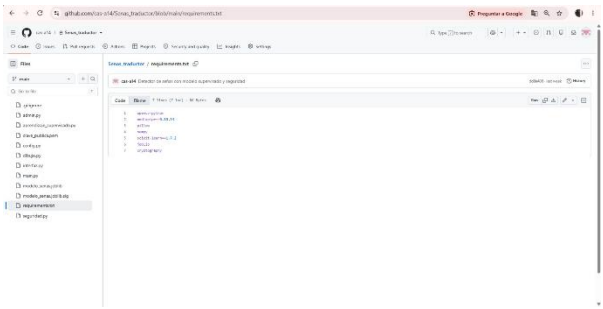


Figura 5. "Lista de librerías necesarias (requirements.txt)", elaboración propia (2026)

5. Lo que no se sube al repositorio

.gitignore le indica a GitHub qué archivos no debe subir, en este caso: los temporales, el archivo de ejemplos del administrador (**dataset_senas.csv**) y todos los archivos **.pem**, excepto la clave pública.

Y así la clave privada del administrador (**clave_privada_admin.pem**) nunca llega al repositorio y se queda solo en la computadora del administrador.

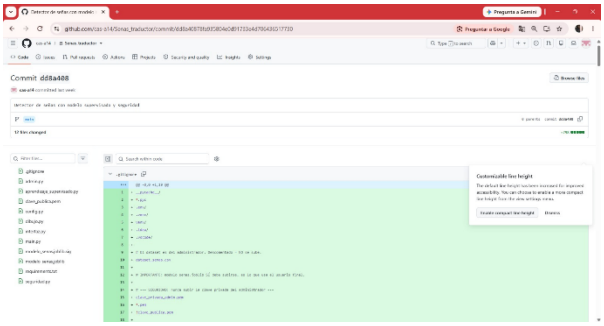


Figura 6. "Archivo .gitignore que protege la clave privada", elaboración propia (2026)

6. Historial de cambios

GitHub guarda cada cambio con un mensaje que explica qué se hizo. En el repositorio aparece el cambio "Detector de señas con modelo supervisado y seguridad" (identificador **dd8a408**) con el que se subieron los 12 archivos del proyecto.

Y gracias a este historial se puede volver a cualquier versión anterior del programa.

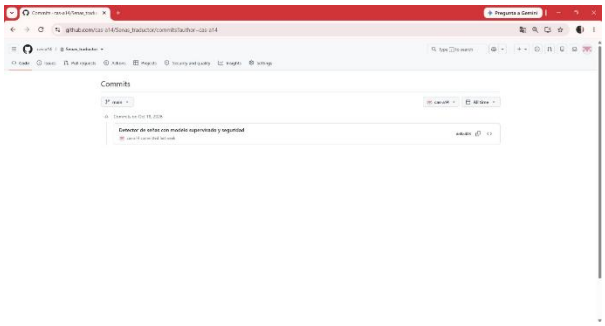


Figura 7. "Historial de cambios (commits) del repositorio", elaboración propia (2026)

Estado del arte

El reconocimiento automático de lengua de señas se estudia desde hace varias décadas. Koller (2020) revisó cerca de 300 trabajos publicados entre 1983 y 2020 y encontró un cambio importante: antes se usaban métodos invasivos, que requerían guantes o sensores, y hoy predominan los métodos basados en video, que no necesitan nada de eso. También comenta que el aprendizaje profundo (una forma de inteligencia artificial que aprende a partir de grandes cantidades de datos) permite trabajar con imágenes completas, aunque todavía se ha puesto poca atención a los rasgos que no son de las manos, como la expresión facial. Rastgoo, Kiani y Escalera (2021) coinciden con esta visión en su revisión de los modelos de aprendizaje profundo de los cinco años anteriores: separan el reconocimiento de señas aisladas del reconocimiento continuo y agrupan los enfoques según usen solo las manos, las manos junto con la cara, o manos, cara y cuerpo.

Otros trabajos buscan que el reconocimiento llegue a los dispositivos que la gente usa todos los días. Alam, Hameed y Ziar (2024) hicieron una revisión sistemática de las técnicas de aprendizaje automático y profundo para detectar lengua de señas en teléfonos inteligentes y concluyeron que estos aparatos ya pueden ejecutar este tipo de modelos y ayudar con la accesibilidad. Algo parecido hicieron Al Khuzayem et al. (2024) con Efhamni, una aplicación para Android que traduce señas aisladas de la lengua de señas saudí a texto y audio en árabe, usando aprendizaje profundo y sin necesidad de un intérprete. Este trabajo nos pareció importante porque casi toda la investigación se centra en lenguas de señas que no son árabes. Por su parte, Maashi, Iskandar y Rizwanullah (2025) propusieron SACHI-SLRHDL, un sistema que une el Internet de las Cosas (dispositivos conectados entre sí) con modelos híbridos de aprendizaje profundo para reducir la brecha de comunicación entre personas sordas y oyentes.

En cuanto a cómo se clasifican las señas, Alsolai et al. (2024) presentan SLDC-RSAHDL, un método que extrae las características de las imágenes con MobileNet, clasifica con una combinación de redes neuronales (CNN y LSTM) y ajusta sus parámetros con el algoritmo de búsqueda del reptil. Los autores mencionan que sus principales retos son las diferencias entre culturas y entre personas, y la necesidad de procesar en tiempo real. En contraste, Carneiro, Salvadeo y de Brito Silva (2024) proponen una opción más ligera: detectan la cara y las manos de quien hace las señas y, a partir

del centro de esas cajas de detección, construyen características sencillas, hechas a mano, que representan la posición y el movimiento de las manos. Con el conjunto de datos AUTSL (lengua de señas turca, con 226 señas y 43 intérpretes) lograron aumentar la exactitud en 7.96 % con muy poco costo computacional adicional, aunque aclaran que todavía no lo han probado en dispositivos portátiles. Este trabajo es el más parecido al nuestro, porque también intenta representar la mano con pocas características en lugar de depender de arquitecturas pesadas.

Por último, hay trabajos enfocados en la salud y en los datos necesarios para entrenar estos sistemas. Camgöz, Kındıroğlu y Akarun (2016a) desarrollaron un sistema de reconocimiento en tiempo real, basado en visión por computadora, para apoyar a personas con discapacidad auditiva en hospitales y así depender menos de los intérpretes. Esos mismos autores, junto con otros colaboradores (Camgöz et al., 2016b), crearon BosphorusSign, un conjunto de datos de lengua de señas turca en los ámbitos de salud y finanzas. Explican que los corpus lingüísticos que ya existían muchas veces no sirven para el reconocimiento automático, porque los métodos estadísticos necesitan ver muchas repeticiones de cada seña y que las cámaras de profundidad económicas hicieron más fácil reunir esos datos.

Análisis y relación con el proyecto

Al revisar estos trabajos vimos que la mayoría de las propuestas recientes usa aprendizaje profundo (redes neuronales como las CNN y las LSTM, o combinaciones de ellas). Esto da buenos resultados, pero exige más recursos de cómputo. Varios de los estudios también señalan como limitaciones la variabilidad entre personas, el procesamiento en tiempo real y la dificultad de llevar estos sistemas a dispositivos de uso diario (Alsolai et al., 2024; Carneiro et al., 2024). Además, notamos que gran parte de la investigación se enfoca en palabras, señas continuas o contextos específicos, y que hay pocos recursos para lenguas de señas poco estudiadas (Al Khuzayem et al., 2024; Koller, 2020).

Nuestro proyecto se distingue por ser sencillo y ligero: reconoce únicamente letras individuales que se forman con la mano quieta. No necesita sensores costosos ni guantes, y sigue la tendencia hacia los métodos basados en cámara que describe Koller (2020). En lugar de un modelo profundo, usa un Random Forest (un conjunto de árboles de decisión que “votan” para elegir

la letra) entrenado con los puntos de la mano. Con esto retoma la idea de Carneiro et al. (2024) de representar la mano de forma compacta y con bajo costo de cómputo, pero con un modelo relativamente sencillo de entrenar. Así, el proyecto ofrece una alternativa ligera y accesible y deja como trabajo futuro el reconocimiento de palabras o frases completas, que es donde se concentran los retos que señalan Rastgoo et al. (2021).

Desarrollo del software

Dicho de forma sencilla, el sistema trabaja en cuatro pasos. Primero, la cámara toma el video (con OpenCV). Segundo, el programa ubica la mano y sus 21 puntos de referencia (con MediaPipe). Tercero, esos puntos se convierten en una lista de números que describe la posición de la mano. Y cuarto, un modelo de aprendizaje supervisado compara esa lista con lo que aprendió y responde con una letra, que se muestra en una ventana de escritorio hecha con Tkinter.

Arquitectura del sistema: cómo está organizado
El programa está dividido en piezas, y cada una se encarga de una tarea:

Pieza	Qué hace
main.py	Abre el programa en modo usuario libre: solo permite detectar letras, usando el modelo protegido con firma.
admin.py	Abre el programa en modo administrador: pide la contraseña (la crea la primera vez que se usa), permite firmar un modelo existente y habilita la grabación de muestras y el entrenamiento.
interfaz.py	Dibuja la ventana con sus botones y paneles y se encarga de que el video fluya sin que la ventana se trabe.
aprendizaje_supervisado.py	Prepara los datos de la mano, guarda las muestras grabadas, entrena al sistema, comprueba qué tan bien aprendió y adivina la letra.
seguridad.py	Maneja la contraseña, las claves y la firma que protegen el modelo.
config.py	Reúne los ajustes generales: rutas de las carpetas, qué tan seguro debe estar el sistema para mostrar una letra, mínimo de muestras necesarias, etc.

Pieza	Qué hace
dibujo.py	Dibuja el recuadro verde y los puntos sobre la mano en la imagen.

Cómo aprende y cómo reconoce el sistema

- **Qué "ve" el sistema.** MediaPipe ubica 21 puntos de la mano y cada uno tiene tres coordenadas, es decir, 63 números en total. Para que el sistema reconozca la seña sin importar en qué parte de la imagen esté la mano ni qué tan cerca esté de la cámara, los puntos se acomodan tomando la muñeca como punto de partida y se ajustan al tamaño de la mano.
- **Ejemplos (dataset).** El administrador graba muestras de cada letra desde la interfaz. Cada muestra se guarda como una fila del archivo dataset_senas.csv, con la letra y los 63 números.
- **Entrenamiento.** Hacen falta al menos 2 letras y 10 muestras por letra (se recomiendan 100 o más). Los ejemplos se reparten en dos grupos: 80% para que el sistema aprenda y 20% para ponerlo a prueba con señas que no ha visto, cuidando que haya de todas las letras en ambos grupos. Con esa prueba se calcula qué porcentaje de aciertos tiene el sistema y se genera un reporte por letra. Al final, el sistema vuelve a aprender con todos los ejemplos y guarda el modelo. Si hay una sesión de administrador, el modelo nuevo se firma.
- **Reconocimiento.** Con cada cuadro de video en el que aparece una mano, el modelo calcula qué tan probable es cada letra. Si la más probable alcanza al menos el **60%**, se muestra esa letra; si no, aparece un signo de interrogación, que significa **"no estoy seguro".**

Estos son los ajustes principales que controlan el comportamiento del sistema:

Ajuste	Valor	Qué significa
UMBRAL_CONFIANZA	0.60	Seguridad mínima (60 %) que debe tener el sistema para mostrar una letra. Por debajo de eso muestra "?".

Ajuste	Valor	Qué significa
MIN_MUESTRAS_POR_LETRA	10	Mínimo de ejemplos por letra para poder entrenar.
NUM_CARACTERISTICAS	63	Cantidad de números que describen la mano: 21 puntos × 3 coordenadas.
min_detection_confidence	0.7	Seguridad mínima (70%) para dar por hecho que lo que ve la cámara es una mano.
max_num_hands	1	Cantidad de manos que se reconocen a la vez: una.

Cumplimiento del alcance

El objetivo del proyecto es reconocer letras individuales del abecedario de la Lengua de Señas Mexicana (LSM) que se forman con la mano quieta, a partir de la posición de la mano frente a una cámara. Para lograrlo, el sistema captura el video con OpenCV, ubica los 21 puntos de referencia de la mano con MediaPipe, identifica la letra con un modelo de aprendizaje automático (Random Forest) y la muestra en una ventana de escritorio con botones para iniciar y detener la detección. En la prueba que se muestra en este documento, el modelo se entrenó con 21 letras: A, B, C, D, E, F, G, H, I, L, M, N, O, P, R, S, T, U, V, W, Y. El reconocimiento de palabras, de frases completas y de letras con movimiento quedó fuera de esta entrega y se plantea como trabajo futuro, como se ha dicho en los apartados anteriores.

Criterios de aceptación

El proyecto se considera aceptado cuando cumple lo siguiente:

- La aplicación abre una ventana con la imagen de la cámara y los botones "Iniciar" y "Detener".
- Al iniciar, el sistema encuentra la mano y dibuja sus puntos sobre la imagen.
- Al formar una letra, el panel "Letra Detectada" muestra esa letra. Si el sistema no está lo bastante seguro (menos del 60%) o no hay mano, muestra "?".
- Al pulsar "Detener", la cámara se apaga y se vuelve a mostrar "?".

- El modelo solo se usa si su firma digital es válida; si no lo es, la aplicación avisa y no lo carga.
- El entrenamiento muestra el porcentaje de aciertos en la prueba y no permite entrenar cuando hay muy pocos ejemplos.
- La clave privada del administrador no aparece en el repositorio.

Facilidad de despliegue

NOTA: Desplegar significa dejar el sistema funcionando en una computadora nueva.

SignAI se puede instalar en cualquier equipo que tenga Python y una cámara web. Como las rutas son relativas a la carpeta del proyecto, funciona igual en cualquier copia del repositorio, sin tener que cambiar nada. Los pasos, que normalmente realiza el personal de sistemas, son estos:

git clone <https://github.com/cas-a14/Senas_traductor> (descarga una copia del proyecto)

cd Senas_traductor (entra a la carpeta del proyecto)

pip install -r requirements.txt (instala las librerías necesarias)

python main.py (abre el programa para el usuario)

python admin.py (abre el programa para el administrador)

El modelo y su firma ya vienen en el repositorio, así que no hace falta entrenar nada para empezar a usarlo. Se recomienda fijar la versión exacta de scikit-learn en requirements.txt, porque un modelo guardado con una versión de la librería puede no abrirse bien con otra distinta.

Documentación del proyecto

El repositorio incluye un archivo README.md, que funciona como un instructivo del proyecto con los pasos para configurarlo. Resumiendo, se instalan las librerías con requirements.txt, se ejecuta python admin.py la primera vez para crear la contraseña, las claves y la firma del modelo, y después se usa python main.py para la aplicación de usuario. Además, cada módulo y cada función del código tiene un comentario que explica para qué sirve, lo que facilita que otra persona pueda mantenerlo.

Control de deuda técnica (calidad y mantenimiento del código)

La deuda técnica son los pendientes que se acumulan cuando se programa con prisa y que cuestan trabajo más adelante. Para evitarla se tomaron estas decisiones:

- Código separado por piezas (configuración, seguridad, aprendizaje, interfaz y dibujo) de modo que cada una se puede revisar o mejorar por separado.
- Ajustes y rutas reunidos en un solo archivo (config.py), sin rutas que dependan de una computadora en particular.
- Manejo de errores al cargar el modelo, reconocer letras y entrenar, para que la aplicación avise del problema en lugar de cerrarse sola.
- El entrenamiento se ejecuta aparte para que la ventana no se congele.
- Un archivo .gitignore que evita subir al repositorio archivos temporales y secretos.

Pendientes que reconocemos: *agregar pruebas automáticas (que revisen el programa sin intervención de una persona), fijar las versiones exactas de todas las librerías y medir de forma formal la fluidez del video, es decir, cuántas imágenes por segundo procesa.*

Calidad del historial de cambios

Cada vez que se guardan cambios en git (el sistema que lleva el historial del proyecto) se escribe un mensaje que explica qué se hizo; por ejemplo: "Detector de señas con modelo supervisado y seguridad". Antes de cada envío se revisa que solo se suban los archivos previstos (el modelo, su firma, la clave pública y el código) y que la clave privada no aparezca. Además, se recomienda activar la verificación en dos pasos en GitHub y proteger la rama principal (main), que es la versión oficial del proyecto, para que nadie la modifique por accidente.

Pruebas de aceptación del usuario

Para comprobar que el sistema hace lo que debe se definieron las siguientes pruebas; cada una indica qué hacer y qué debería pasar, la última columna se llenó al ejecutarlas.

Prueba	Pasos	Resultado esperado	Resultado
Inicio de la app	Abrir el programa (ejecutar main.py)	Se abre la ventana del programa	Aprobada
Inicio de detección	Pulsar "Iniciar" y mostrar la mano	Se ve el video y los puntos de la mano	Aprobada
Reconocimiento	Formar una letra frente a la cámara	El panel muestra la letra correcta	Aprobada
Sin seña clara	Quitar la mano o hacer un gesto que el sistema no conoce	El panel muestra "?"	Aprobada
Detener	Pulsar "Detener"	La cámara se pasma y deja de detectar	Aprobada
Modelo alterado	Modificar una copia del archivo del modelo y abrir la app	La app avisa que la firma no es válida y no carga el modelo	Aprobada
Acceso de administrador	Abrir el modo administrador (admin.py) y escribir mal la contraseña	Aparece: Acceso denegado	Aprobada

Gestión de dependencias y licencias

Una dependencia es una librería que el proyecto necesita para funcionar. Todas las que se utilizan son de código abierto y tienen licencias permisivas o sea que permiten usarlas y distribuirlas siempre que se conserve el aviso de la licencia correspondiente.

Librería	Uso en el proyecto	Licencia
Python (incluye Tkinter, Threading, Queue y Time)	Lenguaje principal y ventana de la aplicación	PSF License

Librería	Uso en el proyecto	Licencia
OpenCV	Captura del video de la cámara	Apache 2.0
MediaPipe (versión 0.10.14)	Detección de la mano (21 puntos)	Apache 2.0
Scikit-learn	Modelo que reconoce las letras (Random Forest)	BSD de 3 cláusulas
NumPy	Manejo de datos y cálculos	BSD de 3 cláusulas
Pillow (PIL)	Mostrar las imágenes en la ventana	HPND (MIT-CMU)
Joblib	Guardar y abrir el modelo	BSD de 3 cláusulas
Cryptography	Claves y firma digital del modelo	Apache 2.0 o BSD

Las dependencias están listadas en el archivo requirements.txt. La versión de MediaPipe ya está fijada y se recomienda fijar también la de scikit-learn.

Seguridad y manejo de datos sensibles

Acceso de administrador. Solo el administrador puede grabar muestras, entrenar y firmar modelos. Para entrar necesita una contraseña de al menos 8 caracteres y tiene 3 intentos para escribirla correctamente. Su clave privada, que es el "sello" con el que firma los modelos, se guarda cifrada con esa misma contraseña.

Integridad del modelo. Los archivos .joblib, donde se guarda el modelo, pueden ejecutar código al abrirse, así que un modelo alterado en el repositorio sería un riesgo real. Para evitarlo cada modelo se firma con la clave privada del administrador, como si se le pusiera un sello y la aplicación comprueba ese sello con la clave pública antes de abrirlo. Además, el modelo se abre a partir de los mismos datos que se verificaron, para que nadie pueda cambiarlos entre la verificación y la carga.

Datos personales. El sistema no guarda fotografías ni video: el archivo de ejemplos contiene únicamente las coordenadas ya ajustadas de los puntos de la mano. Además, el video se procesa en la propia computadora y no se envía a internet.

Repositorio. El archivo .gitignore impide subir la clave privada.

Desempeño y escalabilidad

Para que la ventana no se trabe, la captura y el análisis del video se hacen en segundo plano y siempre se muestra la imagen más reciente de la cámara. La imagen se ajusta a 960 × 720 píxeles y el sistema trabaja con una lista de solo 63 números por seña, por lo que es ligero.

Para agregar letras o mejorar el reconocimiento basta con grabar más muestras y volver a entrenar; el archivo de ejemplos crece de forma proporcional al número de muestras. Como límites actuales, el sistema reconoce una sola mano y letras estáticas (sin movimiento). Las letras que requieren movimiento necesitarían un enfoque distinto, que queda como trabajo futuro junto con el reconocimiento de palabras.

Big data

NOTA. Big data se refiere a volúmenes de información tan grandes, o que llegan tan rápido y en formatos tan variados que no se pueden manejar con herramientas comunes y hacen falta sistemas especiales, como servidores o bases de datos distribuidas.

SignAI no utiliza big data y tampoco lo necesita. Sus datos son un conjunto propio de ejemplos de señas, guardado en un solo archivo CSV (una tabla que incluso se puede abrir en Excel). Cada fila corresponde a una seña grabada y contiene la letra y los 63 números que describen la posición de la mano. En la prueba que se muestra en este documento, el archivo reúne 2,217 muestras de 21 letras, con 100 muestras por letra serían apenas unos miles de filas, algo que la computadora maneja sin problema en memoria con NumPy y el módulo csv.

Además, el sistema no guarda fotografías ni video (como se mencionó anteriormente), solo números, por lo que el archivo es pequeño y no contiene imágenes de las personas. Si en el futuro el sistema creciera, por ejemplo con muchas personas grabando señas al mismo tiempo, muchas más letras o reconocimiento de palabras en video, tal vez sí valdría la pena pensar en una solución de big data. Para esta etapa no fue necesario.

Capturas beta

Este apartado reúne capturas de pantalla de la versión beta, es decir, la versión de prueba del sistema ya funcionando. Sirven para ver, sin instalar nada, cómo luce y cómo responde el programa en uso real. Ojo, las pantallas explicadas paso a paso, con sus botones y mensajes, están en el apartado de experiencia del usuario (UX); aquí se muestran situaciones de funcionamiento que complementan esa explicación.

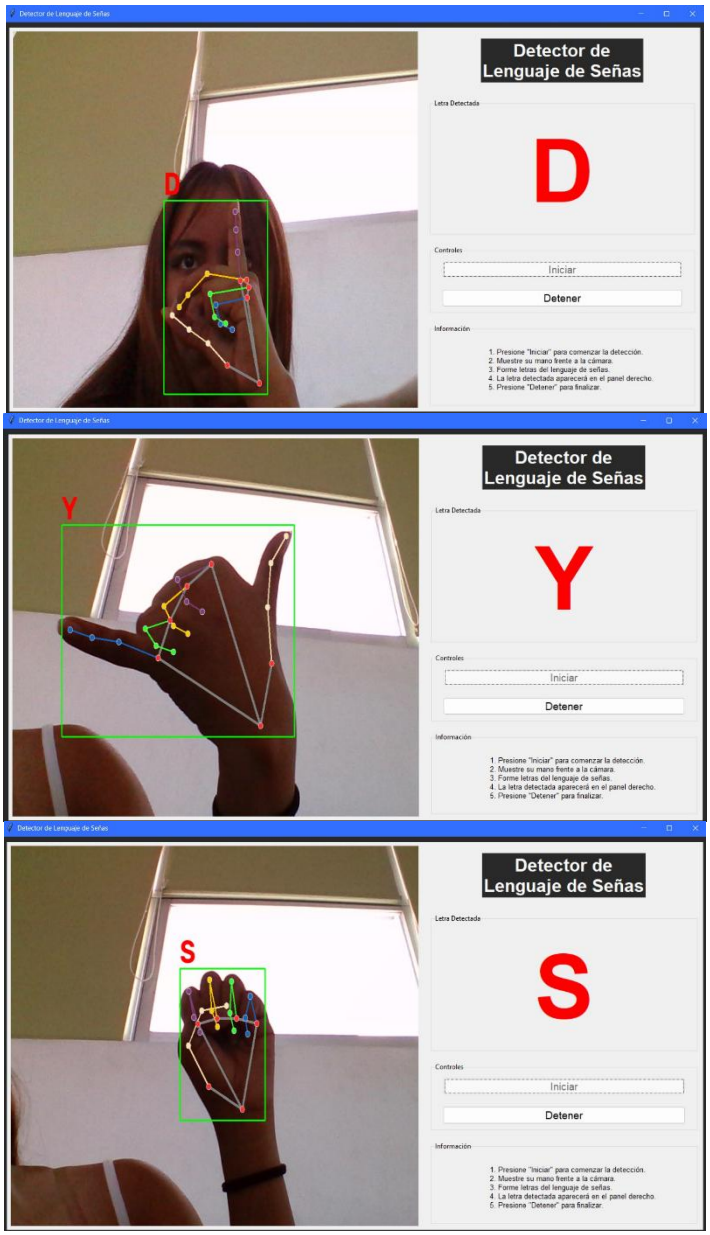


Figura 8. "Reconocimiento de distintas letras", elaboración propia (2026)

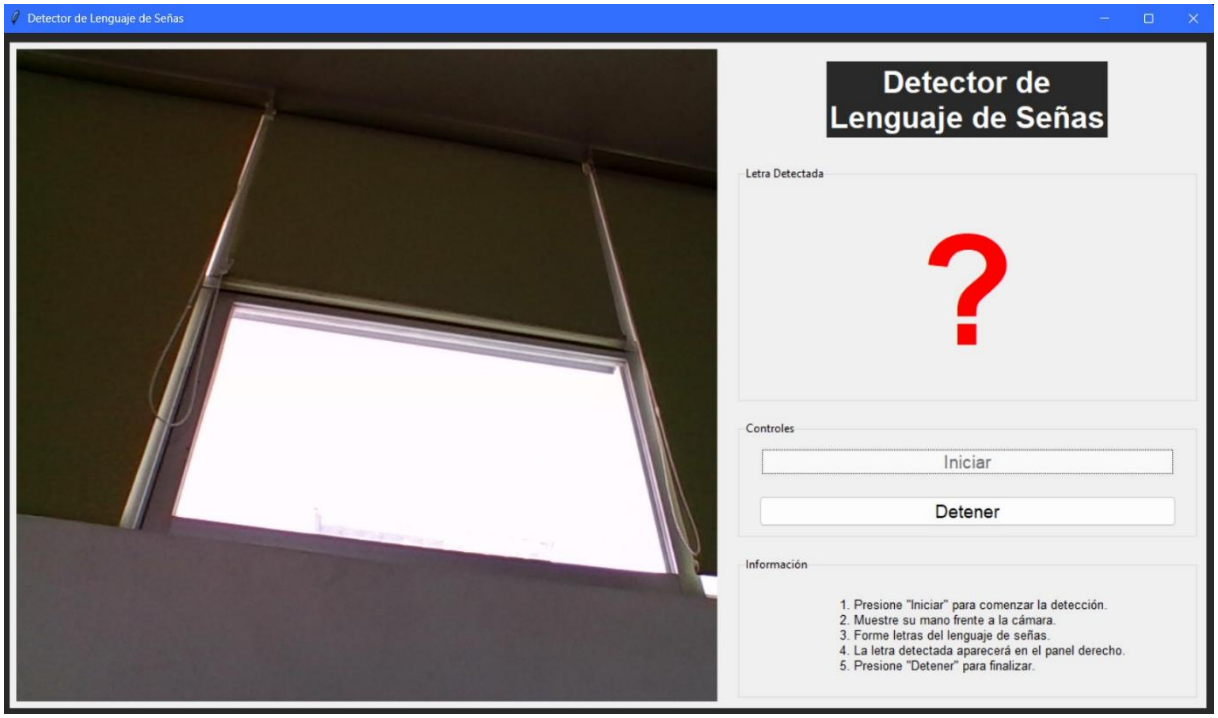


Figura 9. “Pantalla de detección sin una seña clara”, elaboración propia (2026)

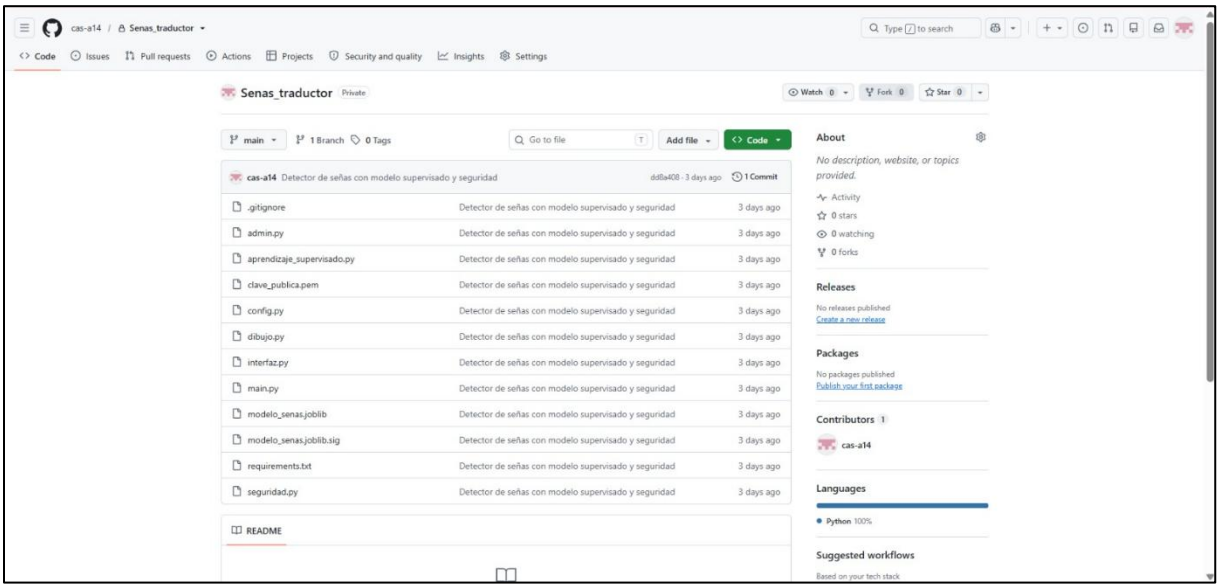


Figura 10. “Repositorio del proyecto en GitHub”, elaboración propia (2026)

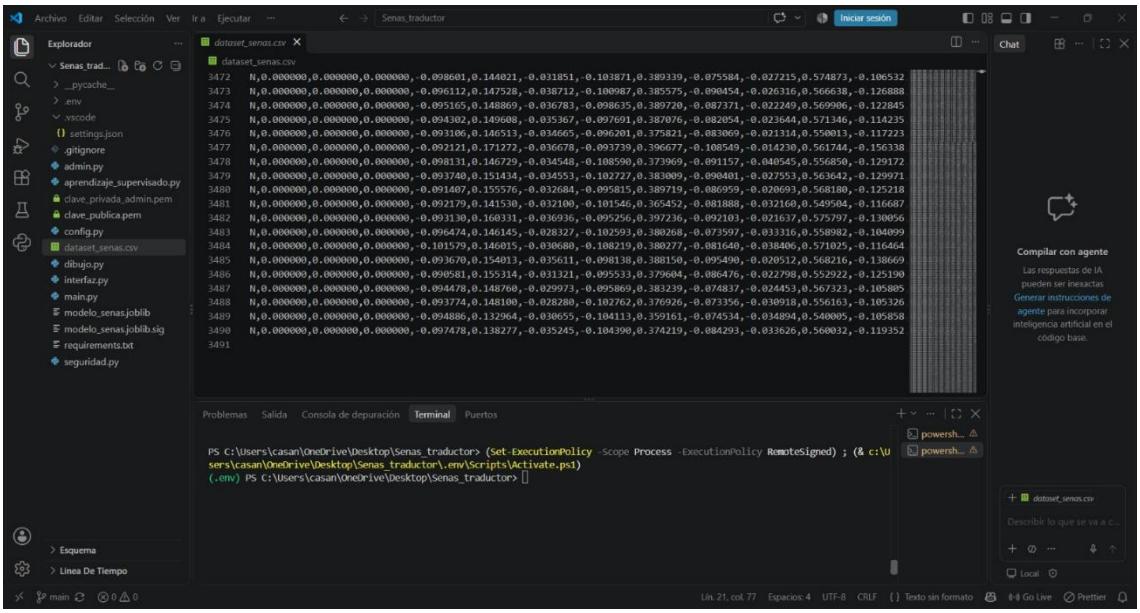


Figura 11. "Dataset generado al grabar muestras", elaboración propia (2026)

UX (Experiencia del usuario)

La experiencia del administrador

1. La puerta de entrada: la contraseña

Lo primero que encuentra el administrador es una ventana pequeña y sencilla que le pide su contraseña. La idea es que las herramientas para enseñar al sistema no queden al alcance de cualquiera, así que solo entra quien la conoce. Si la escribe bien y pulsa “OK“, pasa directo a su panel de trabajo. Si prefiere no continuar y pulsa “Cancel“, la ventana simplemente se cierra sin mensajes ni pasos extra.

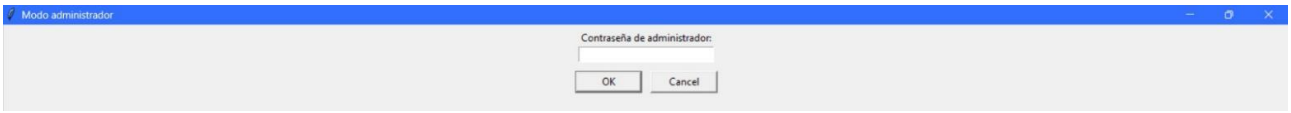


Figura 12. “Pantalla de seguridad del administrador”, elaboración propia (2026)

2. Si la contraseña no es correcta

Si el administrador se equivoca al escribir su contraseña, el sistema no lo deja en blanco ni lo confunde. Aparece una ventana titulada “Acceso denegado“, con un icono rojo y el mensaje “Contraseña incorrecta“, que le avisa con claridad qué pasó. Al pulsar “Aceptar“ la ventana se cierra.

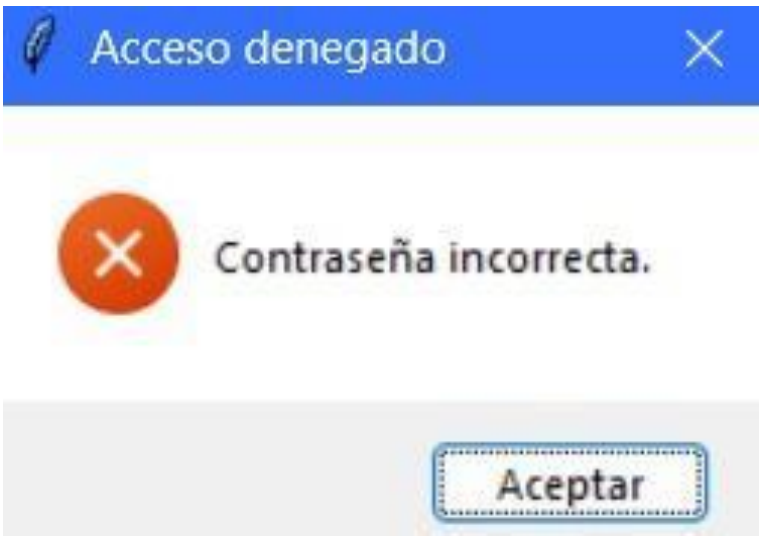


Figura 13. “Ventana de acceso denegado por contraseña incorrecta”, elaboración propia (2026)

3. El panel de administración

Cuando la contraseña es correcta, el administrador llega a su panel, donde encuentra todo reunido en una sola pantalla. A la izquierda está la imagen de la cámara: cuando la mano aparece, el sistema la rodea con un recuadro verde y dibuja encima una especie de esqueleto con los puntos que le sirven para entenderla. En un panel grande a la derecha se muestra la letra que el sistema cree estar viendo.

Debajo están los controles. “Iniciar“ enciende la cámara y “Detener“ la pausa y hace que el sistema deje de buscar letras. Más abajo está la zona de entrenamiento donde el administrador le enseña al sistema casi como se le enseña a una persona: elige de una lista la letra que quiere practicar (por ahora, las que se hacen con la mano quieta), pulsa “Grabar muestras“ y hace esa seña frente a la cámara para que el sistema guarde ejemplos de cómo luce. La pantalla le va avisando cuántas muestras lleva guardadas de esa letra, así que siempre sabe cómo va. Cuando termina, pulsa “Entrenar y evaluar modelo“ para que el sistema aprenda de todos esos ejemplos y, al mismo tiempo, se ponga a prueba.

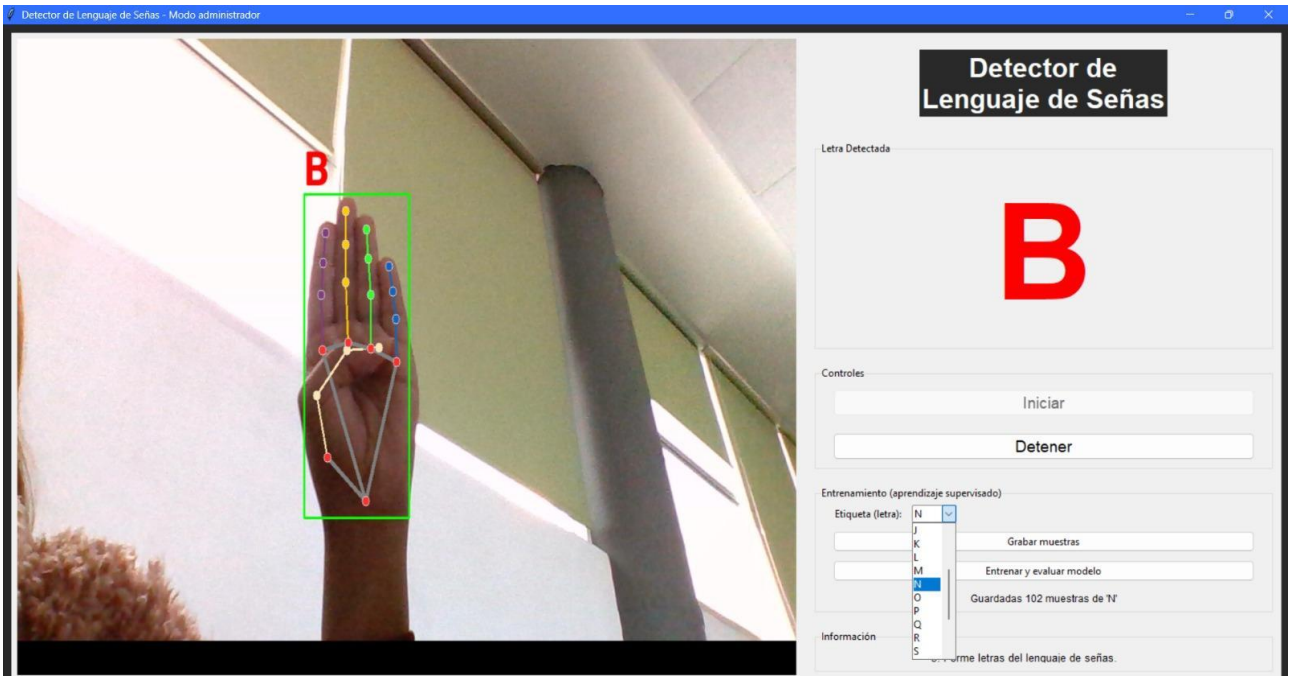


Figura 14. “Panel de administración con cámara, controles y zona de entrenamiento”, elaboración propia (2026)

4. El resumen del entrenamiento

Al terminar el entrenamiento se abre una ventana con el resumen, algo parecido a una boleta de calificaciones. En este ejemplo, el sistema trabajó con 21 letras y 2,217 muestras en total: usó 1,773 para aprender y apartó 444 para examinarse, y en ese examen acertó el 100% de las veces. Ojo, conviene leer ese 100% con cierta cautela: el examen se hace con señas del mismo conjunto de ejemplos grabado (apartadas al azar), así que en el uso diario, con otras personas, otra iluminación o fondos distintos, el porcentaje podría ser menor como se muestra en la figura 10. La ventana también confirma que el modelo quedó guardado en un archivo, para poder usarlo después sin volver a entrenar.

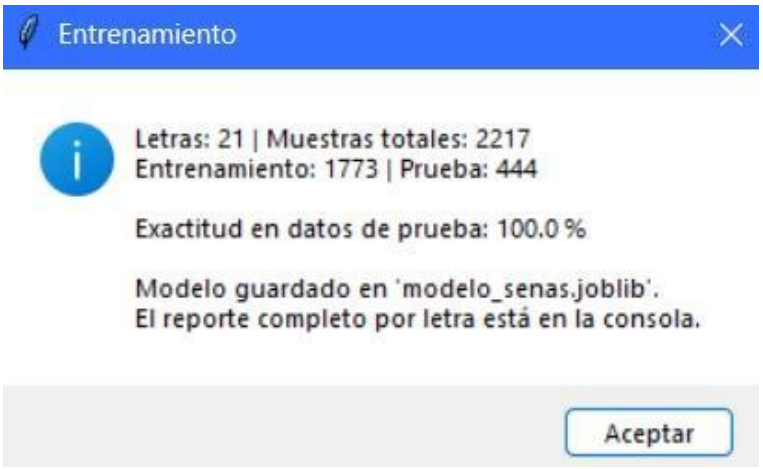


Figura 15. “Ventana de resultados del entrenamiento y evaluación del modelo”, elaboración propia (2026)

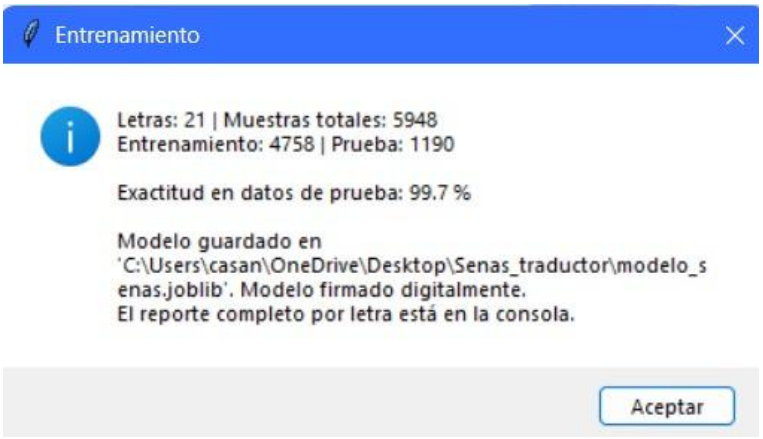


Figura 16. “Ventana de resultados del entrenamiento y evaluación del modelo”, elaboración propia (2026)

La experiencia del usuario

1. Una pantalla que se explica sola

Para el usuario libre todo es más ligero: no hay contraseña, no hay registro y no hay nada que configurar. Abre el programa y entra directo a la pantalla de detección, que solo tiene dos botones, "Iniciar" y "Detener", y que funcionan igual que en el panel del administrador. Abajo hay un recuadro de información con cinco pasos muy cortos, de modo que aunque sea la primera vez que alguien lo usa, no necesita que le expliquen nada.

Al principio, en el panel donde irá la letra se ve un signo de interrogación grande, que quiere decir "todavía no sé qué letra es". La persona pulsa "Iniciar" se enciende la cámara y muestra su mano. El sistema la reconoce, la marca con el recuadro verde y sus puntos, y se prepara para identificar la seña.

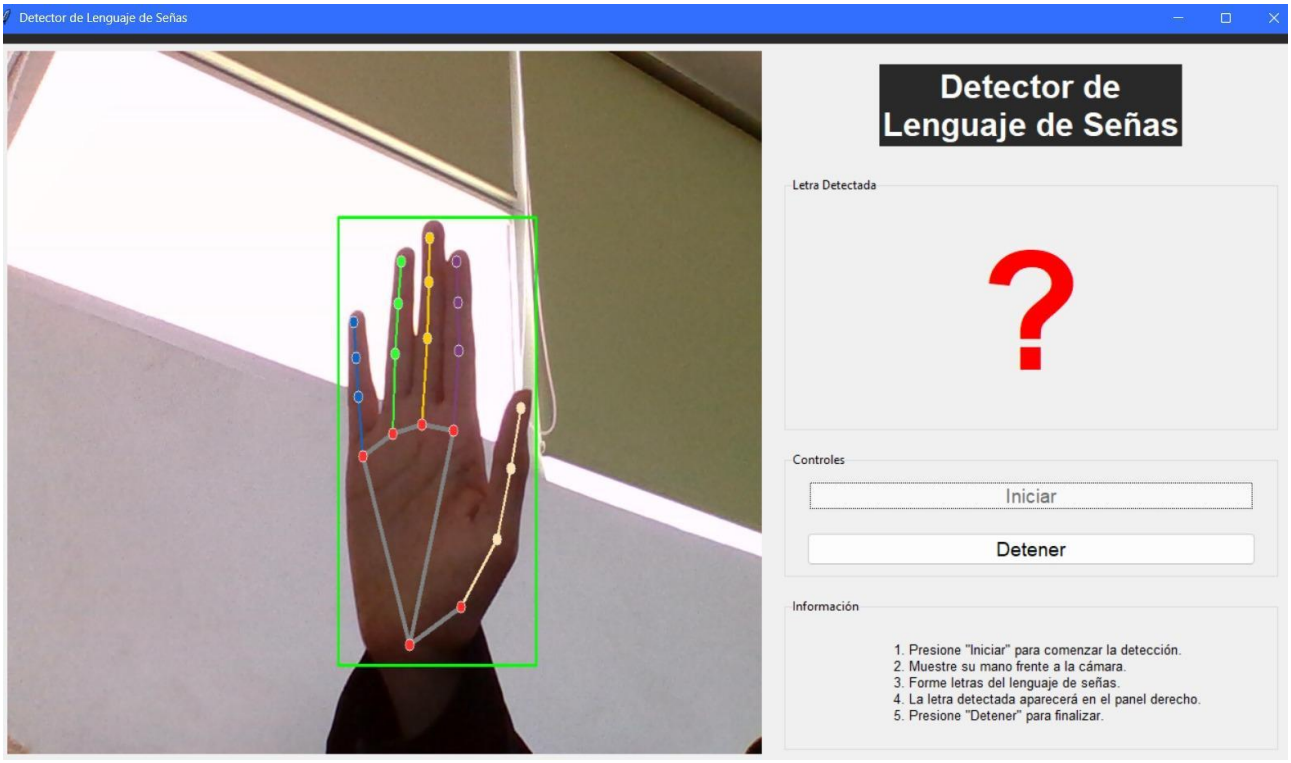


Figura 17. "Pantalla de detección antes de reconocer una letra", elaboración propia (2026)

2. Aparece la letra detectada

En cuanto el sistema identifica la seña, cambia el signo de interrogación por la letra en grande y en rojo y también la muestra sobre la mano en la imagen de la cámara. En el ejemplo aparece la letra B, pero eso no significa que sea la única que el sistema reconoce. Todo ocurre en la misma pantalla, que se va actualizando sola mientras la persona hace sus señas. Cuando termina, pulsa “Detener“ la cámara se pausa y el sistema deja de detectar.

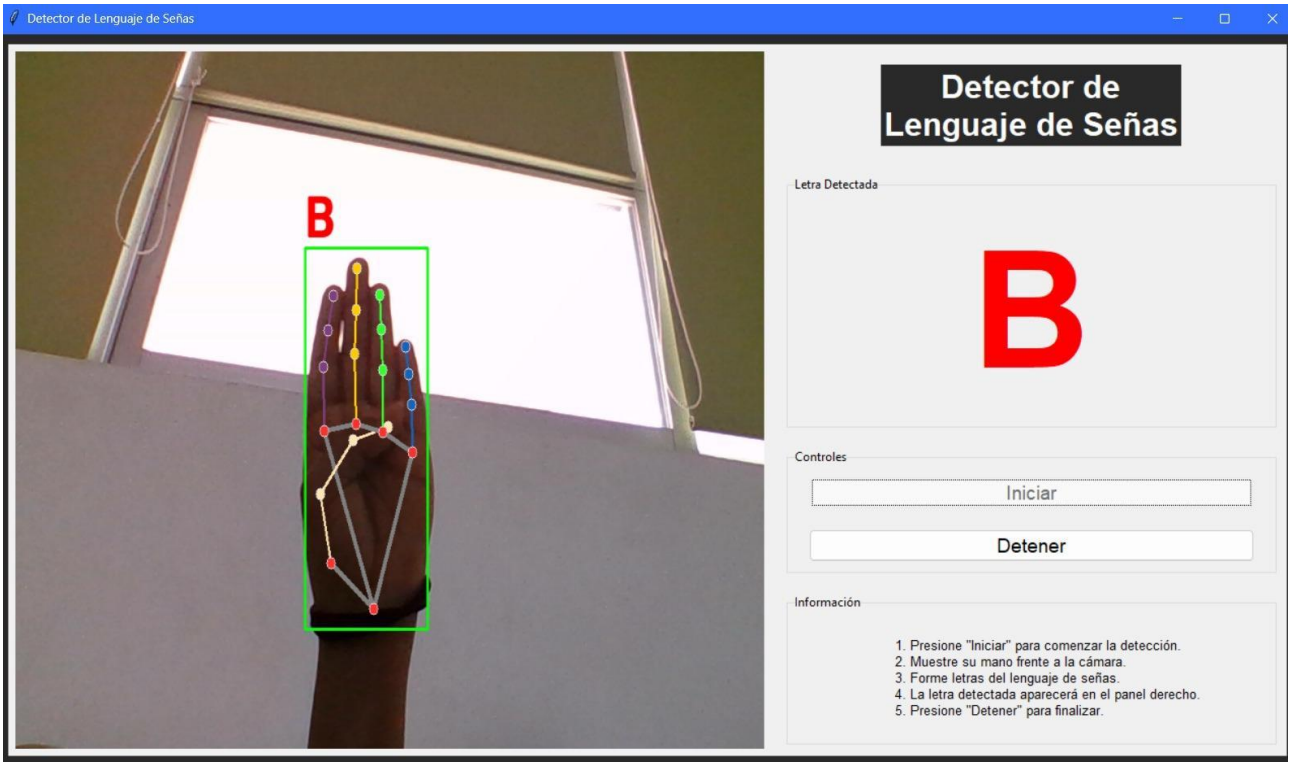


Figura 18. “Pantalla de detección con la letra reconocida (ejemplo: letra B)”, elaboración propia (2026)

Referencias bibliográficas

Visión por computadora aplicada a la salud social (personas sordas y lengua de señas)

Alam, I., Hameed, A., & Ziar, R. A. (2024). Exploring sign language detection on smartphones: A systematic review of machine and deep learning approaches. *Advances in Human-Computer Interaction, 2024*, Article 1487500.

Este artículo es una revisión sistemática sobre las técnicas de aprendizaje automático y aprendizaje profundo que se han usado para detectar e interpretar lengua de señas en teléfonos inteligentes. Los autores parten de que los celulares hoy tienen la capacidad de correr este tipo de modelos y de que pueden ayudar a resolver problemas de accesibilidad.[1] Nos sirve porque resume qué enfoques existen y qué tan viable es llevar el reconocimiento de señas a un dispositivo que las personas sordas ya utilizan todos los días.

Al Khuzayem, L., Shafi, S., Aljahdali, S., Alkhamesie, R., & Alzamzami, O. (2024). Efhamni: A deep learning-based Saudi sign language recognition application. *Sensors, 24*(10), Article 3112.

Las autoras proponen Efhamni, una aplicación móvil para Android que ayuda a las personas sordas y con dificultades auditivas de Arabia Saudita a comunicarse con su comunidad. La app traduce señas aisladas de la lengua de señas saudí a texto y audio en árabe usando un modelo de aprendizaje profundo, sin necesitar un intérprete humano. El trabajo destaca porque casi todas las investigaciones se enfocan en lenguas de señas no árabes, así que este aporte atiende una población poco estudiada.[2]

Alsolai, H., Alsolai, L., Al-Wesabi, F. N., Othman, M., Rizwanullah, M., & Abdelmageed, A. A. (2024). Automated sign language detection and classification using reptile search algorithm with hybrid deep learning. *Heliyon, 10*(1), Article e23252.

El estudio presenta una técnica llamada SLDC-RSAHDL para detectar y clasificar lengua de señas. Usa MobileNet para extraer características de las imágenes, un modelo híbrido que combina redes

convolucionales (CNN) y LSTM para clasificar, y el algoritmo de búsqueda del reptil para ajustar los parámetros. Los autores señalan que el reconocimiento de señas es útil para que las personas sordas se comuniquen con quienes no conocen la lengua de señas, y que sus principales retos son la variabilidad entre culturas y personas y la necesidad de procesar en tiempo real.[3]

Camgöz, N. C., Kındıroğlu, A. A., & Akarun, L. (2016a). Sign language recognition for assisting the deaf in hospitals. In M. Chetouani, J. Cohn, & A. A. Salah (Eds.), *Human behavior understanding* (Lecture Notes in Computer Science, Vol. 9997, pp. 89–101). Springer.

Este trabajo es de los más cercanos al tema de salud, porque desarrolla un sistema de reconocimiento de lengua de señas en tiempo real, basado en visión por computadora, pensado para apoyar a usuarios con discapacidad auditiva en un entorno hospitalario. Muestra un caso concreto en el que la tecnología puede reducir la barrera de comunicación entre pacientes sordos y el personal de salud, que muchas veces depende de intérpretes.[4]

Camgöz, N. C., Kındıroğlu, A. A., Karabüklü, S., Kelepir, M., Özsoy, A. S., & Akarun, L. (2016b). BosphorusSign: A Turkish sign language recognition corpus in health and finance domains. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC 2016)* (pp. 1383–1388). European Language Resources Association.

Los autores presentan BosphorusSign, un conjunto de datos de lengua de señas turca creado para investigar reconocimiento de señas en los ámbitos de salud y finanzas, con la idea de apoyar a las personas sordas en lugares como hospitales y bancos. Explican que los corpus lingüísticos existentes muchas veces no sirven para reconocimiento automático porque los métodos estadísticos necesitan muchas repeticiones de cada seña, y que las cámaras de profundidad económicas permitieron reunir ese tipo de datos.[5] Es útil como ejemplo de recurso de datos enfocado en contextos de salud.

Carneiro, A. L. C., Salvadeo, D. H. P., & de Brito Silva, L. (2024). Sign language recognition based on deep learning and low-cost handcrafted descriptors. *arXiv*.

Propone un sistema eficiente de reconocimiento de lengua de señas que evita sensores costosos y arquitecturas muy pesadas. Entrena un detector de objetos para ubicar la cara y las manos del intérprete y crea características hechas a mano a partir de los centroides de esas cajas, que representan la posición y el movimiento de las manos. Con el conjunto AUTSL (lengua de señas turca, 226 señas y 43 intérpretes) logró aumentar la exactitud en 7.96 % con muy poco costo computacional adicional. Los autores reconocen que aún no lo probaron en dispositivos portátiles, por lo que todavía no está listo para el uso diario de personas sordas.[6]

Koller, O. (2020). Quantitative survey of the state of the art in sign language recognition. *arXiv*.

Es un metaestudio que analiza alrededor de 300 trabajos publicados entre 1983 y 2020 sobre reconocimiento de lengua de señas. Entre sus hallazgos están el cambio de métodos invasivos (guantes o sensores) a métodos no invasivos basados en video, el uso de imágenes completas gracias al aprendizaje profundo y la poca inclusión de rasgos no manuales, como la expresión facial. También señala que RWTH-PHOENIX-Weather es prácticamente el único recurso grande para reconocimiento continuo, y expresa la esperanza de que futuras investigaciones incluyan y sean dirigidas por personas sordas.[7]

Maashi, M., Iskandar, H. G., & Rizwanullah, M. (2025). IoT-driven smart assistive communication system for the hearing impaired with hybrid deep learning models for sign language recognition. *Scientific Reports, 15*, Article 6192.

Este artículo presenta SACHI-SLRHDL, un sistema de comunicación asistiva para personas con discapacidad auditiva que combina el Internet de las Cosas (IoT) con modelos híbridos de aprendizaje profundo para reconocer lengua de señas. Parte de que la lengua de señas es fundamental para que las personas sordas se comuniquen y expresen lo que sienten, y de que estos sistemas ayudan a cerrar la brecha de comunicación con las personas oyentes.[8]

Rastgoo, R., Kiani, K., & Escalera, S. (2021). Sign language recognition: A deep survey. *Expert Systems with Applications, 164*, Article 113794.

Es una revisión de los modelos de aprendizaje profundo aplicados al reconocimiento visual de lengua de señas en los cinco años previos. Propone una taxonomía que separa el reconocimiento aislado del continuo y revisa los enfoques que usan solo las manos, las manos con la cara, o manos, cara y cuerpo. Además, discute conjuntos de datos, aplicaciones y retos. En la introducción menciona que, según la Federación Mundial de Sordos, existen más de 300 lenguas de señas y unos 70 millones de personas sordas que las usan, y que un reto importante es lograr sistemas que funcionen en conversaciones reales.[9]

Fuentes

[1] https://onlinelibrary.wiley.com/doi/10.1155/2024/1487500

[2] https://www.mdpi.com/1424-8220/24/10/3112

[3] https://www.cell.com/heliyon/fulltext/S2405-8440(23)10460-9?_returnURL=https%3A%2F%2Flinkinghub.elsevier.com%2Fretrieve%2Fpii%2FS2405844023104609%3Fshowall%3Dtrue

[4] https://link.springer.com/chapter/10.1007/978-3-319-46843-3_6

[5] https://aclanthology.org/L16-1220.pdf

[6] https://arxiv.org/pdf/2408.07244

[7] https://arxiv.org/pdf/2008.09918

[8] https://www.nature.com/articles/s41598-025-89975-1

[9] https://www.sciencedirect.com/science/article/pii/S095741742030614X?via%3Dihub

Resumen del proyecto

SignAI es una aplicación de escritorio que reconoce letras de la lengua de señas, las que se forman con la mano quieta, a partir de la posición de la mano frente a una cámara. Usa OpenCV para capturar el video y MediaPipe para obtener los 21 puntos de la mano. Esos puntos se ajustan y sirven como datos para un modelo de aprendizaje supervisado (Random Forest). La ventana, hecha con Tkinter, muestra la letra reconocida y permite iniciar y detener la detección.

El modelo aprende de ejemplos grabados por el administrador y se evalúa con señas apartadas para la prueba. Además, el modelo lleva una firma digital y la aplicación solo lo abre si esa firma es válida, lo que protege a las personas usuarias de un archivo alterado.

El estado del arte muestra que la mayoría de los trabajos recientes usa aprendizaje profundo, que exige mucho poder de cómputo. Nuestra propuesta ofrece una alternativa ligera y accesible, pues no necesita sensores ni guantes, solo una cámara web. El reconocimiento de palabras, de frases completas y de letras con movimiento queda como trabajo futuro.